

IN2060 - Digitalteknikk og datamaskinarkitektur

Oblig 1 - Introduksjon til Questa og ARM assembler kompilering

Målet med denne obligen er å bli kjent med verktøy som skal brukes i oblig 2 og 3. Obligen består av to deler. Del 1 er en introduksjon til Questa, som du vil bruke for å simulere HDL (Hardware Description Language) kode i oblig 2. I del 2 skal du bli kjent med et av to verktøy du kan bruke for å kompilere og kjøre ARM assembly kode, enten en Raspberry Pi, eller en emulator. Vanligvis ville alle på kurset gjort oblig 3 på Raspberry Pi, men ettersom det nå kan være vanskelig å få tak i Raspberry Pi er det nødvendig med et digitalt alternativ. Det er derfor satt opp et system for emulering av ARM assembler kode. Emulatoren kan brukes på linux maskinene på ifi, på IFIs [ssh login cluster](#), eller på den virtuelle maskinen Ifi Digital Electronics. Du kan logge inn på VDI ressursen med [VMware Horizon klienten](#), eller i [nettleser](#).

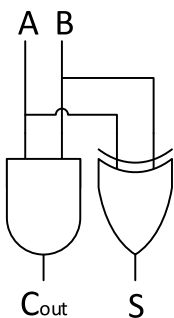
Del 1 - Bli kjent med simuleringsverktøy for digital design

Målet med denne oppgaven er bli kjent med hvordan vi benytter simuleringsverktøy ved utvikling av hardware. Verktøyet vi benytter heter Questa og er et kompilers- og simuleringsverktøy for hardwarespråk (både VHDL og verilog). Skal man kunne analysere og konstruere digitale kretser må man kjenne til hvordan slike verktøy virker og bruke minst ett. Questa er en videreutvikling av Modelsim, som er mye brukt i industrien. De største leverandørene for programmerbar logikk har typisk integrerte varianter av modelsim i verktøyene de tilbyr for utvikling med sine kretser.

Før vi kommer til simuleringen, så må vi få laget en modul som kan kompileres og simuleres.

1. Lag en egen folder på UIO-hjemmeområdet ditt til bruk for denne oppgaven
2. I den første oppgaven har vi kode som skal lagres som en fil med navn halfadder.vhd. Skriv inn programmet nedenfor med et redigeringsverktøy¹ og lagre det i den nye folderen. Det anbefales å skrive teksten manuelt inn i starten, for at oppbyggingen, nøkkelord og syntaksen skal sette seg i fingrene.

¹Et hvilket som helst tekstredigeringsverktøy kan benyttes, bare det kan lagre rene tekstfiler. Det anbefales å bruke verktøy som holder styr på linjenummer, har monospace font, og kan highlighte nøkkelord i VHDL med forskjellige farger. Eksempler kan være VScode, Gedit, Emacs eller Vim. Questa har også en egen editor for dette formålet.



Figur 1 - Halvadder

```
library IEEE; -- bibliotek
use IEEE.STD_LOGIC_1164.all; -- pakke

entity halfadder is
  port(
    a, b : in std_logic; -- inputs
    s : out std_logic; -- sum
    c : out std_logic -- carry
  );
end entity halfadder;
```

```
architecture dataflow of halfadder is
begin
  s <= a xor b
  c <= a and b;
end architecture dataflow;
```

Når du har lagret filen, skal du åpne filen i Questa. Questa er installert på alle linuxmaskinene på ifi. Mer informasjon om Questa er beskrevet på [Questa Sim](#) siden på wikien til robin, <http://robin.wiki.ifi.uio.no>. Hvis du ikke har direkte tilgang til linuxmaskinene på ifi, kan bruke en VMwaremaskin eller ssh til login.ifi.uio.no nærmere beskrivelse finnes på [Remote access](#) og [FPGA tools](#) sidene. Merk at du trenger bare at én av metodene virker for deg.

For å starte Questa, kjører du følgende kommando i terminalen:

```
vsim
```

Dette starter Questas grafiske brukergrensesnitt (GUI). Du kan også starte Questa ved å klikke på ikonet i applikasjonskatalogen.

I Questa:

3. Lag et nytt prosjekt i questa som følger:

- File-> new Project... , velg folderen du lagde i 1), og gi prosjektet et navn.
- Trykk «ok»,
- Velg «Add existing files», og legg til kildefila **halfadder.vhd**. Senere kan du høyreklikke i prosjektvinduet og velge «Add to project» for å legge til eller opprette filer².

4. Kompilér filen («Compile» / «Compile all») og rett eventuelle feil. Kommenter rettingen du gjør i kildekoden.

Merk: Det er normalt at det blir en del feil ved første kompilering. Å kopiere koden for hånd kan lett gi flere feil. I koden til halvadderer mangler det ett semikolon på den tredje siste linjen. Kompilatoren bør finne dette dersom den er kopiert inn eksakt. Ved å dobbeltklikke på filen med feil (markert med rød 'X'), eller den røde teksten i «transcript» vinduet får du frem feillisten. Noen feil kan føre til at mange linjer i koden blir tolket som feil, uten at de nødvendigvis er det. Hindrer en feil kompilatoren i å jobbe videre, kommer andre feil først til syne når man kompilerer på ny. I starten kan det være lurt å rette én feil av gangen, og heller kompilere mange ganger.

²Når man benytter egne filer er det som regel lurt å velge «Reference from current location» når man legger til filer i et prosjekt. Uten unødvendige duplikatfiler er sjansen mindre for at man redigerer én fil og tester en annen.

- Når koden kompilerer skal du simulere: Velg Simulate-> Start simulation. I vinduet som kommer opp velger du «work->halfadder», og trykker på «Optimization Options» (Dersom din versjon av programmet har en opsjon som heter «Enable_optimization» skal denne være skrudd **på**)
- I «Optimization Options» under fanen «Visibility» velg «Apply full visibility to all modules (full debug mode)» og trykk OK og så OK igjen, slik at simuleringsvindue åpnes (tar litt tid).
- Legg til signalene (a, b, s, c) fra halvadderer inn i waveform vinduet. (Velg signalene fra det mørkeblå vinduet, og høyreklikk med musen og velg «add wave»)
- Simuler i 100 nanosekund (skriv «run 100 ns» i transcript vinduet).
- Zoom ut slik at du ser hele waveformen i bildet (Høyreklikk i vinduet med waveforms og velg «zoom full»).

I wave-vinduet, vil du nå se fire røde streker, en for hvert signal, samt at det står U i kolonnen Msgs ved siden av signalnavnene. Det betyr at den gule linja (cursoren) står et sted der signalene er «uninitialized» (derav U). For å få noe vettugt ut av et digitalt design, må vi gi inngangssignalene en verdi.

- Høyreklikk på a-signalet i wave-vinduet, velg «Force» og sett verdien (value) til 0 De andre egenskapene i vinduet trenger du ikke endre på nå.
- Gjør det samme for B signalet
- Kjør 1 mikrosekund til («run 1 us»), zoom fullt ut igjen og flytt cursoren til der signalene er grønne og les av.

På dette tidspunktet bør alle signalene kunne leses av som 0.

13. Test alle mulighetene som følger

- a. Sett så A til 1 og kjør ett mikrosekund,
- b. dernest A til 0 og B til 1 og kjør ett mikrosekund
- c. og så begge til 1 og kjør ett mikrosekund.

14. Zoom ut fullt igjen, og sjekk at du får riktige verdier for sum (s) og carry (c).

15. Lag en bildekopi av Waveformvinduet til rapporten:

- a. Se til at Wave vinduet er aktivt (det aktive vinduet er det du trykket på sist)
- b. I wave-vinduet (eller den øverste menyen til Questa), trykk File->Export->image
- c. Skift bildetype til png
- d. Lagre bildet som halfadder.png. Innleveringen i denne oppgaven skal bestå av bildefilen halfadder.png.

Innleveringen i denne oppgaven skal bestå av bildefilen halfadder.png.

Etterord til Del 1

Det er mange måter å gjøre simuleringer med Questa. Denne oppgaven tar for seg den enkleste og mest grunnleggende måten å gjøre det på. Metoden kan benyttes for å teste i ukesoppgaver og i den neste obligen i kurset.

I den neste obligatoriske oppgaven i IN2060 vil dere også benytte testbenker³ som kan sette signaler automatisk. Å bruke testbenker er som regel raskere enn å teste ting manuelt, men man vil ofte ha behov for begge deler, for eksempel hvis man vil se om en modul gjør det den skal helt i starten, eller teste «en ting til» etter at testbenkkoden er kjørt.

Når vi setter signaler med «force» overskriver vi verdiene signalet hadde fra andre kilder i simulatoren. Hvis man bruker dette på å sette utgangssignaler fra en modul, kan det kamuflere feil som ellers ville vises i simulering. I praksis er dette sjeldent et problem, siden vi bruker testbenker aller mest, men det er greit å være klar over slik at man ikke lurer seg selv.

³En testbenk kan typisk være en VHDL modul som er laget for å sette signaler til en annen modul som man vil teste. Man kan også lage testbenker med andre programspråk eller script, men det er ikke pensum i IN2060

Feilsøking (ved behov)

Hvis du opplever problemer med modelsim som er installert på egen maskin, eller ikke får tak i det, så prøv questa på en virtuell maskin eller via ssh-innlogging på login.ifi.uio.no, før du spør gruppelærer om hjelp. Vi som lager oppgavene kan ikke teste med versjoner du laster ned på egen pc og som krever registrering, og vi vet at tilgangen til nedlastingsversjoner har variert mye de siste årene.

Hvis et menyvalg som står beskrevet i obligteksten er skrudd av (lysegrått og ikke valgbart i menyen), så sjekk om du har riktig undervindu valgt, eller bruk undervindues egne valg dersom det eksisterer.

Med ulike versjoner av modelsim eller questa hender det enkelte ting må settes opp ulikt. Vi vet for eksempel at optimaiseringsinnstillingene (optimization options) har variert gjennom tidene, og ulike feil (bugs) i questa/modelsim har gjort at disse har variert en del. Hvis du ser at du kjører på en eldre versjon enn 2020.4, så kan det hende disse må endres.

Modelsims bruker oppsettsfiler (.ini filer). Hvert prosjekt har sin egen .ini som normalt kopieres fra programmets .ini fil når det opprettes. Hvis man går inn i et gammelt prosjekt så vil de gamle innstillingene gjerne følge med. Dersom et prosjekt er opprettet med feil i .ini filen, så er det oftest greiest å lage et nytt prosjekt med de samme kildefilene, fremfor å fikse det gamle. Et typisk eksempel på dette er hvis man har endret VHDL-versjon i .ini filen etter man opprettet prosjektet: Det går godt an å finne prosjektets .ini fil og rette den, men det er gjerne like raskt å lage et nytt prosjekt.

Del 2 - Bli kjent med ARM assembler kompilering

Når vi koder et program, som skal kjøres av en datamaskin, skriver vi det ofte i et høynivå programmeringsspråk. Høynivå programmeringsspråk er laget for å være enkle å lese og forstå for mennesker. Dette kan for eksempel være Python, Java eller C. Datamaskinen kan ikke direkte forstå og utføre disse programmene. Det er fordi prosessoren(CPU) kun kan utføre et sett instruksjoner som ble definert i hardware da den ble designet. Vi kaller disse instruksjonene for maskinkode. Dere vil lære mer om hva maskinkode er senere i kurset, men for nå holder det å vite all kode vi skriver må oversettes til maskinkode for at datamaskinen skal kunne lese og utføre den.

For noen programmeringsspråk (f.eks. Python) vil koden oversettes til maskinkode av en interpreter underveis mens den kjører, men for mange programmeringsspråk må vi først oversette hele koden til maskinkode før vi kjører den. Programmet som oversetter høynivåkode til maskinkode kalles en kompilator. I denne obligen skal vi compilere og kjøre et C program og et Assembler program.

Hvilket sett av maskinkodeinstruksjoner en prosessor støtter er avhengig av arkitekturen til prosessoren. De to mest vanlige arkitekturer er X86 og ARM. Linux maskinene på ifi har X86 arkitektur, men i dette kurset skal dere kjøre ARM maskinkode. Da må vi enten kjøre koden på en datamaskin som har ARM arkitektur, f.eks. en Raspberry Pi, eller bruke en emulator. ARM emulatoren later som at den utfører ARM maskinkode, men i bakgrunnen må den kjøre maskinkode som samsvarer med arkitekturen til datamaskinen den kjører på.

Denne obligen tar som utgangspunkt at emulatoren blir brukt for å kjøre ARM kode. Dersom du har en Raspberry Pi du vil bruke i stedet kan du følge vår oppsettsguide, og så gå videre til steg 4.

Vi skal nå bli kjent med emuleringsverktøyet ved å compilere og kjøre et C program:

1. For å få tilgang til emulatoren må du enten være logget inn på en av linux maskinene på ifi, logge deg inn på IFIs ssh login cluster, eller logge deg inn på vdi ressursen IFI Digital Electronics. Du kan logge inn på VDI ressursen med VMware Horizon klienten, eller i nettleiser. Dersom du er på en egen mac eller linux maskin kan du følge denne guiden for å logge deg inn med ssh på IFIs ssh login cluster. Dersom du er på en egen windows maskin kan du f.eks. bruke X-Win som beskrevet i denne guiden.
2. Når du har kommet inn på en av maskinene kan du kjøre følgende kommando for å åpne et miljø der du har tilgang på emulatoren:

```
in2060_arm
```

3. Når du vil lukke miljøet kan du gjøre det med følgende kommando:

```
exit
```

4. Vi skal nå compilere og kjøre C kode. Lag en mappe med navn in2060_oblig1, og naviger til mappen i terminalen. I mappen lagrer du koden nedenfor i en fil med navn helloworld.c (*Inne i in2060_arm miljøet finnes kun editorene vim, nano og emacs, og du vil ikke ha tilgang på grafisk brukergrensesnitt. Det kan derfor lønne seg å lagre koden i en fil først, og så åpne miljøet etterpå.*)

```
#include <stdio.h>

int main(int argc, char** argv) {
    printf("Hello World!\n");
    return 0;
}
```

5. Vi skal nå compilere koden. Bruk kommandoen in2060_arm for å åpne miljøet som inneholder emulatoren og kompilatoren. Kompilatoren du skal bruke heter arm-linux-gnueabi-g++. Denne kompilatoren kompilerer til ARM maskinkode.

- a. Kompiler programmet med følgende kommando⁴:

```
arm-linux-gnueabi-g++ -static -o helloworld helloworld.c
```

⁴Vi inkluderer flagget -static til kompilatoren når vi kompilerer koden som skal emuleres. Dette er fordi ARM emulatoren vi bruker er statisk bygget.

- b. Dersom du kompilerer på en RPi eller en annen maskin med ARM arkitektur kan du i stedet bruke gcc til å compilere:

```
gcc -o helloworld helloworld.c
```

6. Du vil nå ha en eksekverbar fil som heter helloworld. Dersom du er på en maskin med ARM arkitektur (en RPi) kan du kjøre programmet med:

```
./helloworld
```

7. Når du prøver å kjøre programmet på denne måten på en maskin med x86 arkitektur, slik som på en ifi maskin, vil du få en *Exec format error*. Denne feilmeldingen forteller deg at programmet du prøver å kjøre er bygget for en annen arkitektur enn den maskinen du kjører på har. Vi kjører derfor koden med *emulatore*n i steden, ved å bruke følgende kommando:

```
qemu-arm-static helloworld
```

8. Koden skal nå kjøre! Ta et skjermbilde av terminalen etter at koden har kjørt og gi det navnet helloworld_c.png. Denne bildefilen er en del av innleveringen til Del 2.

Assembler er et *lavnivå* programmeringsspråk. Dette programmeringsspråket ligger veldig nært maskinkode. Vi kan nesten se på det som en menneskeleselig versjon av maskinkode. Når vi programmerer i Assembler har vi veldig god kontroll på hvilke instruksjoner prosessoren utfører. Dette kan være nyttig hvis man skal lage en kompilator, en driver eller et operativsystem.

Vi går nå videre til neste steg hvor vi skal compilere og kjøre et *Assembler* program:

9. Lagre koden nedenfor i en fil med navn helloworld_assembler.s. Vi bruker endingen .s på filer som inneholder Assembler kode.

```
.text
.global main
main:
    push {lr}

    ldr r0, =string
    bl printf

    mov r0, $0
    pop {lr}
    bx lr

.data
string: .asciz "Hello World!\n"
```

10. For å compilere Assembler koden til maskinkode bruker du følgende kommando (vi har med opsjonen -g for å legge til debuginformasjon til en debugger vi straks skal bruke):

- a. (Husk å åpne miljøet med emulatore

```
arm-linux-gnueabi-g++ -static -g -o helloworld_assembler helloworld_assembler.s
```

- b. Dersom du bruker en RPi kan du igjen bruke gcc i steden for kompilatore

```
gcc -g -o helloworld_assembler helloworld_assembler.s
```

11. Du skal nå ha en eksekverbar fil med navn helloworld_assembler. Kjør den med en av de følgende kommandoene:

- a. Dersom du bruker emulatore

```
qemu-arm-static helloworld_assembler
```

b. Dersom du er på en RPi kjør med:

```
./helloworld_assembler
```

12. Dersom man får en feil når man programmerer i Assembler kan det være vanskelig å finne den ettersom koden ikke er like lett å forstå som høynivå programmeringsspråk. Vi bruker derfor en debugger når vi skal finne feil i Assembler kode. Miljøet med emulatoren har en debugger som heter gdb. Følg disse stegene for å åpne koden din i gdb med et terminalbrukergrensesnitt (tui): (Dersom du er på RPi hopp videre til steg 13)

- a. Først emulerer vi koden. Vi inkluderer -g opsjonen for å videresende debuginformasjon til en port (her port 1234). Dette gjør at gdb kan få tak i informasjon om kjøringen av programmet vårt fra emulatoren. Vi inkluderer & slik at programmet kjører i bakgrunnen. (Merk at du må starte programmet ditt med emulatoren, slik som dette, på nytt for hver gang du vil kjøre det i debuggeren)

```
qemu-arm-static -g 1234 ./helloworld_assembler &
```

- b. Så åpner vi gdb med følgende kommando: (Noen ganger må man trykke på enter en ekstra gang for at gdb skal komme i gang skikkelig. Pass på at det står (gdb) på nederste linje i terminalen før du går videre)

```
gdb-multiarch -tui ./helloworld_assembler
```

- c. Deretter må vi fortelle gdb hvilken port den skal lese fra (pass på at du bruker samme port som du brukte i a):

```
target remote localhost:1234
```

- d. Nå kan vi se hva som ligger i registerene med følgende kommando:

```
tui reg general
```

- e. Registerene er minneelementer der prosessoren har dataene den jobber på (Du vil lære mer om dette senere i kurset). Det er veldig nyttig å se på disse dataene, så prøv å huske på denne kommandoen til oblig 3 :)

13. Dette steget beskriver hvordan vi kjører gdb på RPi. Dersom du bruker emulatoren gå videre til steg 14.

- a. Vi åpner gdb med følgende kommando: (Noen ganger må man trykke på enter en ekstra gang for at gdb skal komme i gang skikkelig. Pass på at det står (gdb) på nederste linje i terminalen før du går videre)

```
gdb -tui ./helloworld_assembler
```

- b. Nå kan vi se hva som ligger i registerene med følgende kommando:

```
tui reg general
```

14. Ta et skjermbilde av gdb der man kan se hva som ligger i registerene og lagre det i en fil med navn gdb_tui.png. Denne bildefilen er en del av innleveringen til Del 2.

15. Du kan nå undersøke hvordan gbd fungerer. Test følgende kommandoer:

1. r Kjører programmet (ps. legg inn et stoppested (breakpoint) før du kjører programmet) (pps. hvis du bruker emulatoren kjører allerede programmet når du åpner gdbtui, på RPi må du starte det selv)
2. s Kjører neste instruksjon
3. break main Legger inn et stoppested ved *main*: i koden
4. c Kjører til neste stoppested, eller til slutten av koden dersom det ikke er noen stoppesteder
5. q Avslutter debuggingsprogrammet

16. Når du kjører koden kan du se at verdiene i registerene endrer seg.

17. Vi skal nå gjøre en liten endring i programmet `helloworld_assembler.s`, før vi kjører det på nytt. Mot slutten av assembler koden finner du følgende instruksjon:

```
mov r0, $0
```

18. Denne instruksjonen flytter tallet 0 til registeret `r0`. Legg til ny kode rett før instruksjonen `mov r0, $0` som i stedet flytter tallet 5 til `r0` og tallet 3 til `r1`.

19. Når tallene er flyttet til `r0` og `r1` skal vi legge dem sammen, vi kan gjøre det med instruksjonen:

```
add r0, r1
```

20. Resultatet av addisjonen vil du kunne se i registeret `r0`.

21. Kompiler den modifiserte koden, og kjør den med debuggeren `gdb-tui`. Gå steg for steg igjennom koden, og ta et skjermbilde av debuggeren rett etter at `add` instruksjonen har blitt utført. (Slik at man kan se resultatet i `r0`)

22. Den modifiserte assemblerfilen `helloworld_assembler.s` og bildefilen `gdb_tui_add.png` er en del av innleveringen til Del 2.

Innlevering oblig 1:

Fra Del 1:

- `halfadder.png`

Fra Del 2:

- `helloworld_c.png`
- `gdb_tui.png`
- `helloworld_assembler.s`
- `gdb_tui_add.png`